E05 — Swap Nodes in Pairs (Linked List)

Experiment: 5 | **LeetCode:** #24 | **CO:** CO2 | **BT Level:** BT3

Problem Statement

Given a linked list, swap every two adjacent nodes and return its head.

You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed).

Example 1:

```
Input:  1 → 2 → 3 → 4
Output: 2 → 1 → 4 → 3
```

Example 2:

```
Input:  []
Output: []
```

Example 3:

```
Input:  [1]
Output: [1]
```

Node Definition

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def build_list(values):
    if not values:
        return None
    head = ListNode(values[0])
    curr = head
    for v in values[1:]:
        curr.next = ListNode(v)
        curr = curr.next
    return head

def list_to_array(head):
    result = []
    while head:
        result.append(head.val)
        head = head.next
    return result
```

Solution 1: Iterative (with Dummy Head)

Concept

Use a **dummy head** to simplify edge cases. Use a `prev` pointer to re-link pairs.

```
dummy → 1 → 2 → 3 → 4

Step 1: Swap nodes 1 and 2:
  first = 1, second = 2
  prev.next = second        dummy → 2 → ...
  first.next = second.next   1 → 3 → ...
  second.next = first        2 → 1 → 3 → 4
  prev = first (move to 1)

Step 2: Swap nodes 3 and 4:
  first = 3, second = 4
  Similar → 4 → 3

Final: dummy → 2 → 1 → 4 → 3
```

```
def swapPairs_iterative(head):
    """
    Iterative swapping with dummy node.
    Time: O(n)  |  Space: O(1)
    """
    dummy = ListNode(0)
    dummy.next = head
    prev = dummy

    while prev.next and prev.next.next:
        first = prev.next
        second = prev.next.next

        # Swap the pair
        prev.next = second
        first.next = second.next
        second.next = first

        # Move prev to the end of the swapped pair
        prev = first

    return dummy.next
```

Detailed Iterative Trace

Input: `1 → 2 → 3 → 4`

```
Initial:
  dummy → 1 → 2 → 3 → 4
```

```
  prev = dummy

Iteration 1:
  first = node(1), second = node(2)

  Step A: prev.next = second
          dummy → 2 → 3 → 4    [1 is now detached from dummy]

  Step B: first.next = second.next
          1 → 3 → 4             [1 points to what was after 2]

  Step C: second.next = first
          2 → 1 → 3 → 4

  Full list: dummy → 2 → 1 → 3 → 4
  prev = node(1)

Iteration 2:
  prev.next = node(3), prev.next.next = node(4)
  first = node(3), second = node(4)

  Step A: prev.next (i.e., node(1).next) = node(4)
          dummy → 2 → 1 → 4
  Step B: first.next = second.next = None
          3 → None
  Step C: second.next = first
          4 → 3 → None

  Full list: dummy → 2 → 1 → 4 → 3
  prev = node(3)

Iteration 3:
  prev.next = None → loop ends

Return dummy.next = node(2)
Result: 2 → 1 → 4 → 3 ✓
```

Solution 2: Recursive

```
def swapPairs_recursive(head):
    """
    Recursive swapping.
    Time: O(n)  |  Space: O(n/2) — recursion stack depth
    """
    # Base case: 0 or 1 node
    if not head or not head.next:
        return head

    first = head
    second = head.next
```

```
    # Recursively swap the rest of the list
    first.next = swapPairs_recursive(second.next)
    second.next = first

    return second   # New head of this pair
```

Recursive Trace for `1 → 2 → 3 → 4`

```
swapPairs(1→2→3→4)
  first=1, second=2
  first.next = swapPairs(3→4)
                 first=3, second=4
                 first.next = swapPairs(None) = None
                 second.next = 3
                 return 4   → 4→3→None
  second.next = 1
  return 2   → 2→1→4→3→None ✓
```

Odd-Length List Trace

Input: `1 → 2 → 3`

```
dummy → 1 → 2 → 3
prev = dummy

Iteration 1:
  first=1, second=2
  Swap: dummy → 2 → 1 → 3
  prev = node(1)

Iteration 2:
  prev.next = node(3), prev.next.next = None
  Condition: prev.next.next is None → loop ends

Result: 2 → 1 → 3 ✓   (last odd node unchanged)
```

Test Suite

```
def test_swap_pairs():
    solutions = [swapPairs_iterative, swapPairs_recursive]

    test_cases = [
        ([1, 2, 3, 4], [2, 1, 4, 3]),
        ([], []),
        ([1], [1]),
        ([1, 2, 3], [2, 1, 3]),
        ([1, 2], [2, 1]),
    ]

    for fn in solutions:
```

```
        print(f"\n{fn.__name__}:")
        for vals, expected in test_cases:
            head = build_list(vals)
            result = fn(head)
            result_arr = list_to_array(result)
            status = "PASS" if result_arr == expected else "FAIL"
            print(f"  {status}: {vals} → {result_arr}")

test_swap_pairs()
```

Complexity Analysis

Solution	Time	Space	Notes
Iterative	O(n)	O(1)	Preferred — constant space
Recursive	O(n)	O(n/2)	Simpler code, stack overhead

Key Takeaways

- **Dummy head** simplifies pointer manipulation by avoiding special cases for the head node.
- **Pointer manipulation order matters:** change `prev.next` first, then `first.next`, then `second.next`.
- **Recursive approach:** elegant and matches the problem's recursive structure — but uses O(n) stack space.
- Common pattern for linked list problems: use 3 pointers (`prev`, `first`, `second`) + dummy head.

References

- LeetCode #24 — Swap Nodes in Pairs
- Cormen et al., *Introduction to Algorithms*, Ch. 10.2 (Linked Lists)